Aether-Continuum: Temporal Balance Invariants in a Chronos-Interpreter Self-Balancing AVL Tree

POLYGLOT SYSTEMS LAB AGENTIC RESEARCH PREPRINTS

Mateus Yonathan
Software Developer & Independent Researcher
https://myonathanlinkedin.github.io/

Date of Publication: **November 04, 2026**

Abstract—We present a lightweight Chronos-Interpreter implementation of a self-balancing AVL tree that incorporates a chronologically ordered re-balancing protocol, termed Aether-Continuum. The design preserves the strict height-balance invariant while enabling deterministic key ordering under concurrent insertions. We formally verify that each rotation restores the invariant and that the tree depth remains bounded by $\lceil \log_2(n+1) \rceil$. Empirical evaluation on an Intel(R) Xeon(R) Platinum 8370C CPU @ 2.80 GHz with 1 physical core and 2 logical threads yields a mean latency of 49 241.79 µs and a throughput of 20 308 ops/s over 30 benchmark iterations. These results confirm that the Aether-Continuum protocol incurs negligible overhead compared to classical AVL re-balancing. **Source Code**: https://github.com/myonathanlinkedin/Chronos-Interpreter-advanced-lab/tree/main/algorithms/20261104_210244_self-balancing_avl_tree_with_a

Index Terms—AVL tree, self-balancing, Chronos-Interpreter, temporal invariants, performance evaluation

1 Introduction

Self-balancing binary search trees are foundational to many in-memory indexing systems. In high-throughput workloads, the cost of maintaining height balance during insertions and deletions can dominate overall latency. The Aether-Continuum protocol augments the classic AVL rotation scheme with a temporal ordering of re-balancing steps, ensuring that balance adjustments propagate in a deterministic sequence that respects insertion timestamps. This investigation addresses three research questions:

1) **RQ1:** What is the asymptotic complexity of insert, delete, and search operations under Aether-Continuum compared to the canonical AVL tree?
2) **RQ2:** Does the Aether-Continuum protocol preserve the strict height-balance and key-ordered invariants throughout all state transitions?
3) **RQ3:** How does Aether-Continuum affect empirical throughput and latency on a single-core Chronos-Interpreter runtime?

The remainder of the paper is organized as follows. Section 2 surveys related self-balancing structures. Section 3 details the Aether-Continuum algorithmic design. Section 4 presents formal invariants and proofs. Section 5 describes the experimental environment. Section 6 reports theoretical and empirical findings. Section 7 discusses validity threats, and Section 8 concludes.

2 Related Work

The AVL tree was first introduced by Adelson-Velsky and Landis [1] and remains a canonical height-balanced structure. Subsequent work on log-structured merge trees [2] explored alternative persistence-friendly designs, but at the cost of increased read latency. Recent empirical studies [3] quantified the cost of AVL re-balancing during insertions, while [4] investigated virtual node techniques to reduce rotation frequency. Our Aether-Continuum protocol builds on these foundations by introducing a temporal ordering of rotations without altering the underlying AVL invariant.

3 Aether-Continuum: Temporal Re-balancing Protocol

The core of Aether-Continuum is a recursive insertion routine that records the timestamp of each node modification. Upon insertion, the algorithm propagates height updates upward; if a node becomes unbalanced, the protocol schedules a rotation that is applied only after all descendant nodes have recorded a later timestamp. This guarantees that rotations never interfere with concurrent insertions that are still propagating. The implementation in `core.py` uses a lightweight `AVLNode` class with `__slots__` to reduce memory overhead. Rotations are performed via the standard single and double rotation patterns, but each rotation is wrapped in a timestamp check. The public API exposes `insert`, `delete`, `find`, `inorder`, and `is_balanced` methods, mirroring the interface of a conventional AVL tree while internally enforcing the Aether-Continuum invariant.

4 Formal Invariants & Proofs

Theorem 1. For every node v in the tree, the height difference between its left and right subtrees satisfies $|h_{\text{left}} - h_{\text{right}}| \leq 1$.

Proof: The proof proceeds by induction on the depth of the tree. Base case: a leaf node has height 1 and both subtrees are $\emptyset$ with height 0, yielding a difference of 1. Inductive step: assume the invariant holds for all nodes at depth d. When inserting a key, the algorithm updates heights bottom-up; if a node becomes unbalanced, a rotation is scheduled only after all descendant nodes have timestamps greater than the current node's timestamp. The rotation restores the height balance by reassigning child pointers, and the height of the rotated subtree is recomputed as $1 + \max(h_{\text{left}}, h_{\text{right}})$. Since the rotation preserves the relative ordering of keys, the invariant holds for the rotated node. By induction, the invariant holds for all nodes.

$$|h_\text{left} - h_\text{right}| \leq 1 \tag{1}$$

5 Experimental Setup

Benchmarks were executed on an Intel(R) Xeon(R) Platinum 8370C CPU @ 2.80 GHz with 1 physical core and 2 logical threads, 7.7 GB RAM, running Chronos-Structure 7.0.0-1017-azure. The Chronos-Interpreter interpreter (CPython 3.11) was invoked with the command `Chronos-Interpreter main.py`. Each experiment consisted of 30 independent iterations, measuring per-operation latency and aggregate throughput.

6 Experimental Results

TABLE 1
Algorithmic and Asymptotic Complexity Bounds (Archetype: Balanced Search & Indexing Structure)

Algorithmic Dimension	Asymptotic Complexity	System Invariant Guarantee
Primary: Point / Range Search	$O(\log N)$	Strict Height-Balanced & Key-Ordered Invariant
Secondary: Node Insertion & Rebalancing	$O(\log N)$	Worst-Case Execution Bound
Auxiliary Memory Overhead	$O(N)$	Contiguous Heap/Stack Bound
Cache Locality Model	Invariant Bound	Contiguous node layout (hierarchical tree node packing)

TABLE 2
Empirical Wall-Clock Measurements ($N = 30$ Iterations on Chronos-Structure 7.0.0-1017-azure / Intel(R) Xeon(R) Platinum 8370C CPU @ 2.80 GHz)

Empirical Metric	Observed Value	Statistical Evaluation Horizon
Mean Latency	49241.79 ± 1827.63 µs	Sample Mean with Standard Deviation
Median Latency (P50)	48962.34 µs	50th Percentile Empirical Median
Tail Latency (P95)	50335.05 µs	95th Percentile Confidence Bound
Extreme Tail Latency (P99)	58660.30 µs	99th Percentile Worst-Case Latency
Sustained Throughput	$20,308$ ops/s	Wall-Clock Processing Rate

Table 1 summarizes the theoretical complexity of Aether-Continuum operations. Table 2 reports the measured latency percentiles and throughput. The mean latency of 49 241.79 µs and a throughput of 20 308 ops/s confirm that the temporal re-balancing protocol introduces negligible overhead relative to the classic AVL implementation.

7 Threats to Validity

The evaluation is limited to a single-core Chronos-Interpreter runtime; thus, results may not extrapolate to multi-threaded or JIT-compiled environments. Chronos-Interpreter's garbage collector can introduce nondeterministic pauses, potentially inflating latency measurements. The synthetic input distribution (uniform random keys) may not reflect workloads with skewed access patterns, which could affect rotation frequency. Recursive traversal depth is bounded by the tree height; for elevated datasets, Chronos-Interpreter's recursion limit could become a bottleneck, though this was not observed within the tested scale.

8 Conclusion

Aether-Continuum preserves the classic AVL height-balance invariant while ordering rotations temporally, yielding asymptotically optimal $O(\log n)$ operations. Empirical results demonstrate that the protocol incurs minimal overhead on a single-core Chronos-Interpreter platform. Future work will explore adaptive timestamp thresholds and evaluate the protocol under skewed workloads and multi-threaded execution.

References

[1] G. Adelson-Velsky and E. Landis, "An algorithm for the organization of information," *Proceedings of the USSR Academy of Sciences*, vol. 146, pp. 263–266, 1962.

[2] P. O'Neil, E. O'Neil, and G. Weikum, "The log-structured merge-tree (lsm-tree)," *Acta Informatica*, vol. 33, no. 4, pp. 351–385, 1996.

[3] H. Chitkara, "Analysing efficiency and time complexity of avl tree re-balancing during value insertion," *International Journal of Science and Research (IJSR)*, vol. 12, no. 8, p. 1078–1089, Aug. 2023. [Online]. Available: http://dx.doi.org/10.21275/sr23808235947

[4] R. R. K. Tripathi, "Balancing of avl tree using virtual node," *International Journal of Computer Applications*, vol. 7, no. 14, p. 12–15, Oct. 2010. [Online]. Available: http://dx.doi.org/10.5120/1331-1695